

■ City-Wide AI Traffic Engine

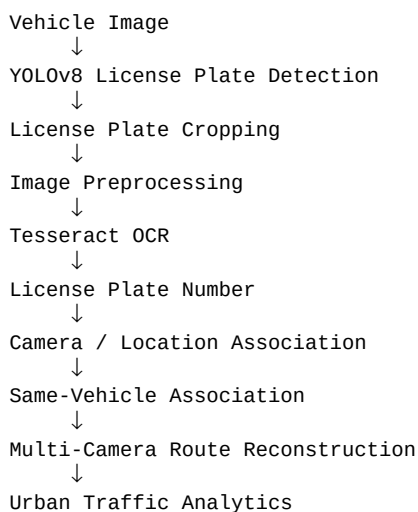
Multi-Camera ANPR • Vehicle Trajectory Tracking • Urban Traffic Analytics

A Python-based prototype for city-wide Automatic Number Plate Recognition (ANPR) using YOLOv8 and Tesseract OCR, extended with simulated multi-camera vehicle association, route reconstruction, and session-based traffic analytics.

Current prototype: The system uses uploaded vehicle images rather than live CCTV/RTSP streams. Each uploaded image is assigned to a simulated camera/location, allowing the project to demonstrate how the same vehicle can be associated across multiple camera observations.

■ Overview

Traditional ANPR systems generally focus on detecting a license plate from a single camera image. This project extends that idea toward a city-wide traffic intelligence engine.



■ Key Features

■ Automatic Number Plate Recognition

- Detects license plates using a trained YOLOv8 model.
- Crops detected plate regions.
- Applies image preprocessing before OCR.
- Extracts plate text using Tesseract OCR.
- Cleans OCR output into an alphanumeric vehicle registration identifier.
- Supports multiple detected plates in a single image.

■ Multi-Camera Simulation

Camera	Location
CAM-01	Suchitra Junction
CAM-02	Kukatpally
CAM-03	JNTU Road
CAM-04	Miyapur

■ Multi-Camera Vehicle Association

The license plate number is used as the primary vehicle identity. For example, TG257602 can be observed at CAM-01 and later at CAM-03, allowing the prototype to reconstruct a route such as Suchitra Junction → JNTU Road.

■ Urban Traffic Analytics

- Total vehicle observations
- Unique vehicles
- Active cameras
- Multi-camera vehicles
- Camera-wise activity
- Vehicle observation frequency
- Multi-camera routes
- Traffic activity level

Important: The current prototype calculates traffic activity from ANPR observations in the image-upload session. It does not claim to measure real-time road congestion.

■■ Interactive Dashboard

- Camera/location selection
- Single-image upload
- Multiple-image upload
- AI detection
- Image preview
- Detection results
- Multi-camera trajectory display
- Urban traffic analytics
- Detection event history
- Output folder access
- Session data reset

■ Technologies Used

Technology	Purpose
Python	Core programming language
YOLOv8 / Ultralytics	License plate detection
OpenCV	Image processing
Tesseract OCR	License plate text recognition
Pytesseract	Python interface for Tesseract
CustomTkinter	Desktop GUI
Pillow	Image loading and preview
Pandas	Detection/history data handling

Technology	Purpose
CSV	Lightweight detection database

■ Project Structure

```

SIH ANPR/
■
■■■ models/
■   ■■■ best.pt
■
■■■ output/
■   ■■■ cropped_plate_*.jpg
■   ■■■ preprocessed_plate_*.jpg
■
■■■ venv/
■
■■■ detector.py
■■■ gui.py
■■■ database.py
■■■ history.py
■■■ requirements.txt
■■■ vehicle_database.csv
■■■ README.md

```

File Responsibilities

gui.py — Main dashboard: upload, camera selection, detection, association, trajectories, analytics.

detector.py — YOLOv8 inference, plate bounding boxes, cropping, preprocessing, OCR and text cleaning.

database.py — Persistent detection records using CSV.

history.py — Detection history, vehicle history, camera history, unique vehicles, trajectories and statistics.

models/best.pt — Trained YOLOv8 license-plate detection model.

output/ — Generated plate crops and preprocessed plate images.

■■ Installation

1. Install Python

Install Python 3.12.x on Windows and verify:

```
python --version
```

2. Create a virtual environment

```
python -m venv venv
.\venv\Scripts\python.exe --version
```

3. Install Python dependencies

```
.\venv\Scripts\python.exe -m pip install -r requirements.txt
```

■ Tesseract OCR Installation

Tesseract is a separate Windows application. Installing pytesseract alone is not enough. The project expects Tesseract at:

```
C:\Program Files\Tesseract-OCR\tesseract.exe
```

Verify the installation with:

```
& "C:\Program Files\Tesseract-OCR\tesseract.exe" --version
.\venv\Scripts\python.exe -c "import pytesseract; print(pytesseract.get_tesseract_version())"
```

■ Running the Application

```
.\venv\Scripts\python.exe gui.py
```

■ How to Use the Prototype

- 1. Select a Camera** — Choose a simulated camera/location from the dropdown.
- 2. Upload an Image** — Use Upload Image or Upload Multiple Images. Clear vehicle photographs with visible plates are recommended.
- 3. Run AI Detection** — YOLOv8 detects the plate, OpenCV preprocesses it, and Tesseract OCR extracts the plate number.
- 4. Test Multi-Camera Association** — Process the same plate at different simulated cameras to reconstruct an observed route.

Example Multi-Camera Demonstration

```
Image 1
TG257602
CAM-01 — Suchitra Junction
    ↓
Image 2
TG257602
CAM-03 — JNTU Road
    ↓
Route:
Suchitra Junction → JNTU Road
```

■ Understanding the Analytics

Total Observations — Number of valid ANPR observations in the current session.

Unique Vehicles — Number of distinct license plate numbers detected.

Active Cameras — Number of camera IDs that generated observations.

Multi-Camera Vehicles — Vehicles observed across more than one camera.

Camera Activity — Number of ANPR observations generated by each camera.

Vehicle Observation Frequency — Number of observations for each identified vehicle.

Multi-Camera Movement — Camera/location sequence reconstructed for vehicles seen at multiple cameras.

Traffic Activity — Prototype activity indicator based on the number of ANPR observations in the current image session.

```
0 observations → NO DATA
1-2 observations → LOW
3-5 observations → MODERATE
6+ observations → HIGH
```

This is a demonstration metric and is not a calibrated real-world congestion measurement.

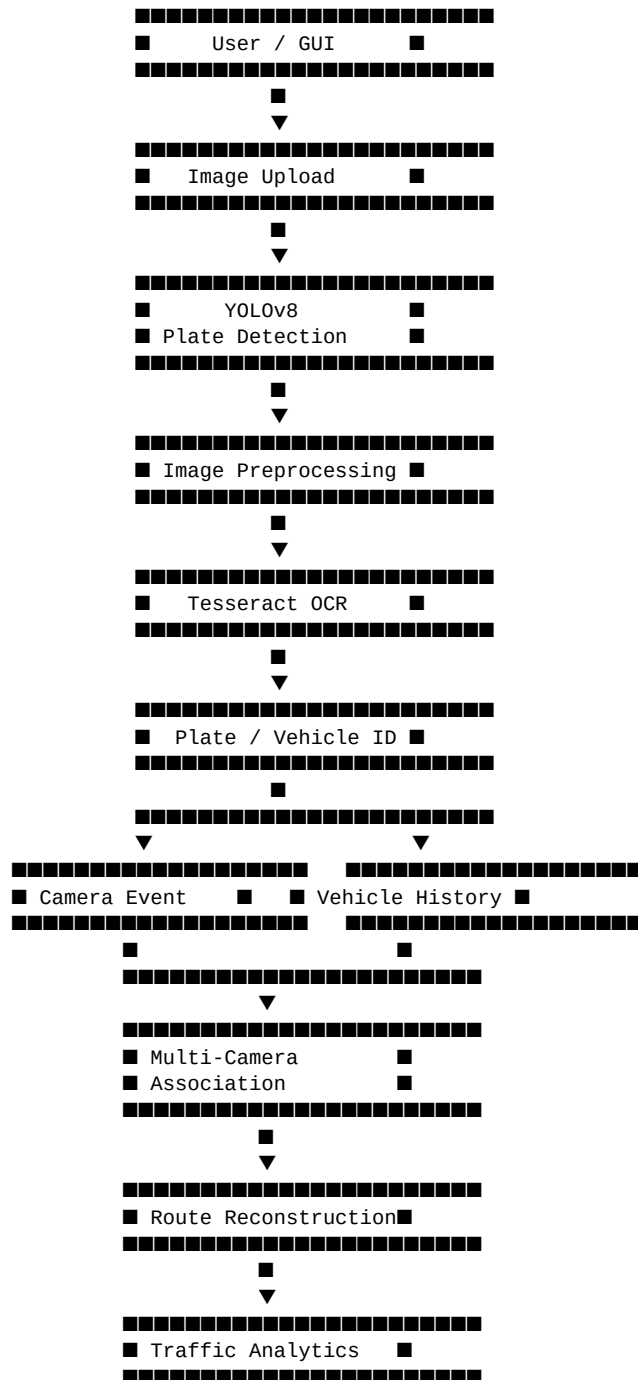
■ Recommended Input Images

- Clear vehicle photographs
- Front or rear vehicle views
- Visible license plates
- Reasonable lighting
- Sufficient plate resolution

- Minimal motion blur

The current image mode is especially suitable for demonstrating ANPR, vehicle identity, multi-camera association and route reconstruction. For genuine traffic-density analytics, future versions should use road/junction images containing multiple vehicles.

■ ■ Current System Architecture



■ Future Enhancements

Real-Time CCTV / RTSP — Replace image uploads with live camera streams.

Vehicle Tracking — Add object tracking such as ByteTrack or BoT-SORT.

Real Traffic Density — Use road-scene images/video to estimate vehicle counts, classes, lane occupancy, queue length and congestion indicators.

City-Wide Route Visualization — Display camera-to-camera vehicle routes on a map.

Cloud / Central Database — Move from local CSV storage to a centralized database.

Advanced Analytics — Peak periods, vehicle flow, travel time, route frequency, congestion prediction and anomaly detection.

Alert System — Potential alerts for authorized blacklists, stolen vehicles, route anomalies and traffic buildup.

■ Prototype vs Production

Capability	Prototype	Future Production
License plate detection	✓	✓
OCR	✓	✓
Image upload	✓	—
Multiple image processing	✓	—
Camera simulation	✓	—
Multi-camera association	✓	✓
Route reconstruction	✓	✓
Session analytics	✓	—
Live CCTV	—	✓
Real-time tracking	—	✓
True traffic density	Limited	✓
City map visualization	—	✓
Cloud database	—	✓

■■ Troubleshooting

Tesseract not installed — Check C:\Program Files\Tesseract-OCR\tesseract.exe and run the Tesseract version command.

YOLO model not found — Ensure models/best.pt exists inside the project folder.

Python package errors — Run .\env\Scripts\python.exe -m pip install -r requirements.txt.

Application does not start — Run .\env\Scripts\python.exe gui.py from the project directory and inspect the terminal error.

■ Privacy and Responsible Use

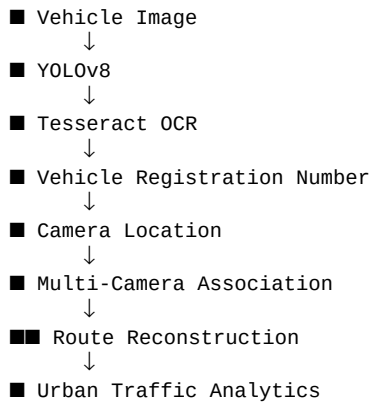
ANPR systems process vehicle registration information, which can be sensitive in real deployments. A production implementation should consider appropriate authorization, data retention policies, access control, secure storage, audit logging, applicable privacy and surveillance regulations, and responsible use of vehicle/location information. This project is currently a demonstration prototype using locally supplied images.

■ Project Objective

The long-term objective is to develop a city-wide AI traffic intelligence platform capable of combining observations from multiple cameras to understand vehicle movement and urban traffic patterns.

Detect → Recognize → Identify → Associate → Reconstruct → Analyze

■ Demo Summary



■ Final Statement

City-Wide AI Traffic Engine is a prototype for intelligent, multi-camera traffic monitoring that combines ANPR, OCR, vehicle identity association, route reconstruction, and traffic analytics into a single desktop dashboard. The current system focuses on demonstrating the architecture and workflow using uploaded images, while providing a clear foundation for future real-time CCTV-based smart-city deployment.